ENSA SAFI · FILIÈRE GIIA

Rapport de TD-TP HTML5

Portail étudiant : activités et ressources

Éléments avancés HTML5 et création d'une page Web

RÉALISÉ PAR

RACHAD MUSTAPHA
AZIZ YASSINE

MODULE	Système de Gestion de Contenu (CMS)
ÉTABLISSEMENT	ENSA Safi
FILIÈRE	GIIA
ANNÉE UNIVERSITAIRE	2026-2027
DURÉE DU TD-TP	6 heures

Sommaire

1 Présentation générale du projet 3

2 Architecture et charte graphique 3

3 Les sections de la page, en détail 5

4 Tests et liste de vérification 13

5 Démonstration : réponses aux questions 14

6 Difficultés rencontrées et conclusion 16

1 Présentation générale du projet

Ce rapport accompagne la page Web « Portail étudiant : activités et ressources », réalisée dans le cadre du TD-TP HTML5 du module Système de Gestion de Contenu. L'objectif du travail est de comprendre et d'utiliser plusieurs éléments HTML5 avancés pour construire une page structurée, accessible et fonctionnelle : tableaux, formulaires, iframe, audio, vidéo, canvas et éléments complémentaires.

Le portail rassemble les rendez-vous de la vie étudiante (conférences, ateliers, compétitions, forum des stages), permet de consulter le planning des clubs de l'école, de s'inscrire à une activité et d'accéder à des ressources utiles. Toute la page tient dans un unique fichier `index.html`, accompagné d'un dossier `media/` pour les fichiers audio et vidéo.

Exigences du mini-projet et élément qui y répond

Exigence du sujet	Réalisation dans la page
Structure sémantique	`header`, un bouton de menu, `nav` (menu tiroir), `main`, sept `section`, `footer`
Présentation	Titre principal, texte d'introduction et liste d'activités mise en forme en quatre cartes
Tableau (au moins 4 activités)	Planning de 4 clubs : activité, date, lieu, responsable
Formulaire (au moins 6 types de champs)	text, email, tel, date, radio, select, checkbox, textarea, datalist
Intégration	iframe OpenStreetMap avec un marqueur sur l'ENSA Safi, et liens de secours
Multimédia	Lecteur audio et lecteur vidéo (media/extrait.mp3, media/presentation.mp4)
Canvas (au moins 3 éléments)	Rectangle, cercle, ligne, triangle et texte
Éléments supplémentaires (au moins 3)	details / summary, progress, meter, output, address, time, datalist
Navigation	Menu tiroir ouvert par un bouton fixe : 7 liens reliés chacun à un identifiant existant

2 Architecture et charte graphique

Structure du document

La page respecte la structure HTML5 recommandée. Le document déclare `<!DOCTYPE html>` pour activer le mode standard, la langue française avec `<html lang="fr">` (utile aux lecteurs d'écran et à la correction orthographique), l'encodage UTF-8 pour les caractères accentués et la balise viewport pour l'affichage sur mobile.

```
<!DOCTYPE html>
<html lang="fr">
<head> <meta charset="UTF-8"> <meta name="viewport" ...> <title>...</title> ... </head>
<body>
  <header> ... </header>
  <button class="menu-btn" id="menuBtn" aria-expanded="false" aria-controls="menu"> ... </button>
  <div class="overlay" id="overlay"></div>
  <nav class="drawer" id="menu" aria-label="Navigation principale"> ... </nav>
  <main> <section id="presentation"> ... </section> ... </main>
  <footer> ... </footer>
</body>
```

Charte graphique

Le design s'inspire de l'identité visuelle du Maroc : un motif de zellige discret en filigrane dans l'en-tête et dans le menu, un bleu nuit profond, un bleu vif pour les actions et un safran pour les accents. Les couleurs sont déclarées une seule fois sous forme de variables CSS (`:root`), ce qui garantit la cohérence de toute la page.

Bleu nuit	Bleu vif	Safran	Fond	Vert
`#0E2A47`	`#2446C9`	`#F2A900`	`#F3F5F9`	`#1F8A5B`
Titres, en-tête, menu, pied de page	Liens et actions	Accents, boutons, focus	Arrière-plan de la page	Confirmation, forme du canvas

Typographie. Les titres utilisent la police Sora et le texte courant Source Sans 3 (Google Fonts), avec des polices système de secours (Segoe UI, system-ui) si le chargement échoue. Les captures de ce rapport ont été réalisées avec les polices de secours.

Mise en page des sections. Chaque section commence par un intitulé centré : un grand titre au dégradé bleu nuit vers bleu vif, une phrase d'accroche et un ornement (filet safran avec losange central) dessiné en SVG intégré dans le CSS.

Adaptation aux écrans. La mise en page est fluide : sous 820 px, les grilles (cartes, formulaire, compléments) passent sur une colonne et les marges se réduisent. Le tableau est placé dans un conteneur à défilement horizontal pour ne jamais déborder. Le bouton de menu reste fixé en haut à droite pendant le défilement et le tiroir occupe au plus 88 % de la largeur de l'écran. Un `scroll-padding-top` évite que les titres soient collés au bord après un clic sur un lien d'ancre, et la règle `prefers-reduced-motion` supprime les transitions pour les personnes qui le demandent.

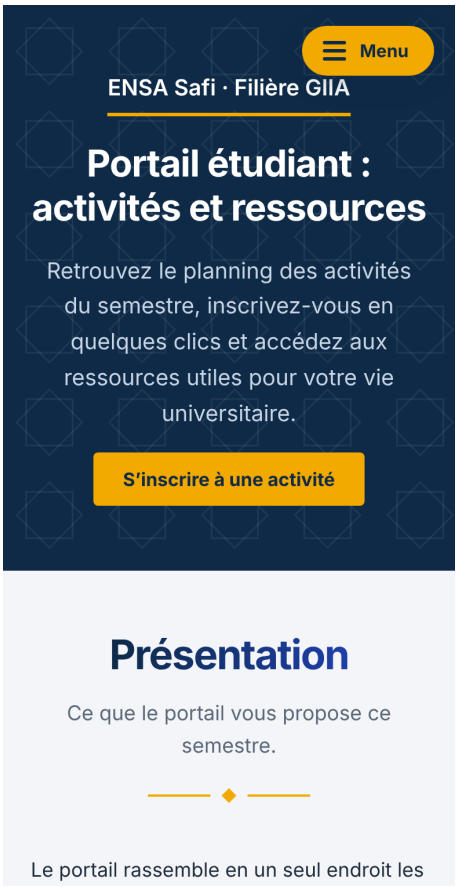


Figure 1 – Affichage de la page sur un écran de téléphone (390 px de large).

3 Les sections de la page, en détail

Pour chaque section : son rôle, les éléments HTML utilisés, un extrait de code et le résultat obtenu.

3.1 En-tête et menu tiroir

L'`header` présente la page : un repère (« ENSA Safi · Filière GIIA »), le titre principal `h1` — unique dans le document —, une phrase d'introduction et un bouton d'appel à l'action qui mène directement au formulaire. Le fond reprend le motif de zellige, dessiné par une image SVG intégrée dans le CSS.



Figure 2 – En-tête avec motif de zellige, bouton d'inscription et bouton « Menu » fixe.

La navigation se fait par un **menu tiroir**. Un bouton `button.menu-btn` en forme de pastille safran, fixé en haut à droite (`position: fixed`), ouvre et ferme le menu. Son icône de « burger » est dessinée en CSS (un `span` et ses pseudo-éléments `::before` et `::after`) et se transforme en croix quand le menu est ouvert. Le bouton porte `aria-expanded` (état ouvert ou fermé) et `aria-controls` (identifiant du menu contrôlé), et son texte passe de « Menu » à « Fermer ».

Le `nav.drawer` glisse depuis la droite grâce à `transform: translateX(100%)` ; tant qu'il est fermé, `visibility: hidden` empêche d'atteindre ses liens au clavier. Un calque `overlay` assombrit le reste de la page, et le défilement de la page est bloqué pendant l'ouverture (`overflow: hidden`). Les sept liens d'ancre (`href="#planning"`, etc.) sont numérotés automatiquement par des compteurs CSS (`counter-increment` et `counter(lien, decimal-leading-zero)`), sans numéro écrit dans le HTML ; l'attribut `aria-label` distingue ce menu d'une éventuelle autre navigation pour les technologies d'assistance.

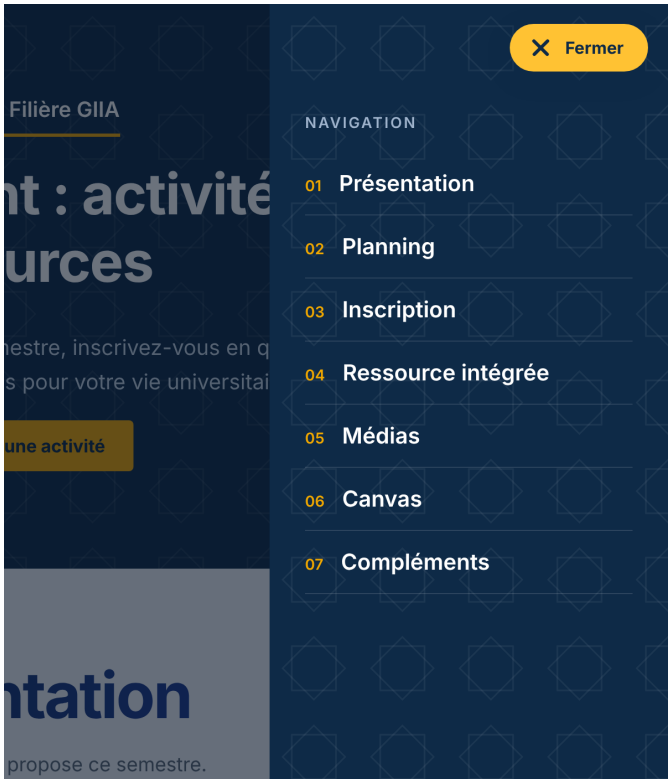


Figure 3 – Menu tiroir ouvert : fond assombri, liens numérotés et bouton « Fermer ».

Un court script gère le comportement : le menu se ferme au clic sur le fond, au clic sur un lien et avec la touche Échap, qui rend aussi le focus au bouton.

```
function definir(ouvert) {
  document.body.classList.toggle("menu-open", ouvert);
  btn.setAttribute("aria-expanded", ouvert);
  txt.textContent = ouvert ? "Fermer" : "Menu";
}
btn.addEventListener("click", function () { definir(!document.body.classList.contains("menu-open")); });
document.getElementById("overlay").addEventListener("click", function () { definir(false); });
document.addEventListener("keydown", function (e) { if (e.key === "Escape") { definir(false); btn.focus(); } });
```

3.2 Section « Présentation » (id="presentation")

Cette section répond à l'exigence de présentation : un texte d'introduction qui décrit le rôle du portail, puis une **liste d'activités** (`ul` / `li`) mise en forme en quatre cartes par CSS Grid (`repeat(auto-fit, minmax(200px, 1fr))`) : conférences, ateliers, compétitions et carrières. Le liseré supérieur des cartes alterne bleu vif et safran grâce au sélecteur `li:nth-child(even)`.

```
<ul class="activites">
  <li><strong>Conférences</strong>Des intervenants du monde professionnel présentent leurs métiers.</li>
  <li><strong>Ateliers</strong>Des séances pratiques pour progresser en développement web.</li>
  <li><strong>Compétitions</strong>Hackathons et défis de programmation par équipes.</li>
  <li><strong>Carrières</strong>Forum des stages et ateliers de préparation aux entretiens.</li>
</ul>
```



Figure 4 – Section « Présentation » : introduction et liste d'activités en cartes.

3.3 Section « Planning » (id="planning")

Le planning est un vrai tableau de données : un tableau est adapté ici, car l'information est organisée en lignes et en colonnes. Il comporte un titre (`caption`), une ligne d'en-tête (`thead` avec des `th scope="col"`), un corps (`tbody`) et quatre activités de clubs : club sportif, club social, club ENIAC (informatique) et club Robotech (robotique). Le nom du club est un `th scope="row"` : il sert d'en-tête de ligne. Les dates utilisent l'élément `time` avec un attribut `datetime` lisible par les machines. Le tableau est enveloppé dans un conteneur `.defile` (`overflow-x: auto`) qui lui permet de défiler horizontalement sur un petit écran.

```
<div class="defile"><table>
  <caption>Planning des activités des clubs, semestre 1 (2026-2027)</caption>
  <thead><tr><th scope="col">Activité</th><th scope="col">Date</th>
             <th scope="col">Lieu</th><th scope="col">Responsable</th></tr></thead>
  <tbody>
    <tr><th scope="row">Club sportif</th>
        <td><time datetime="2026-11-18">18 novembre 2026</time></td>
        <td>Terrain de sport</td><td>Bureau du club</td></tr>
    ...
  </tbody>
</table></div>
```

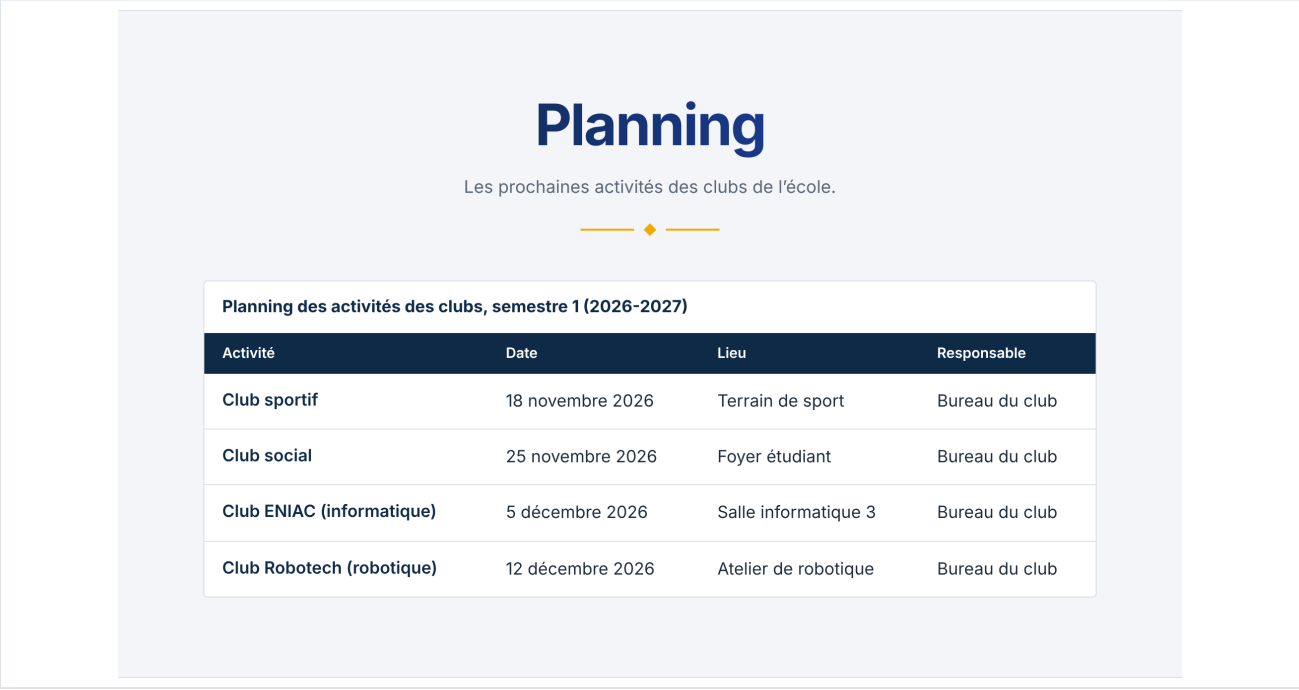


Figure 5 – Tableau du planning (en-tête sombre, survol des lignes en surbrillance).

À retenir. `th` est une cellule d'en-tête (texte en gras, annoncée comme telle par les lecteurs d'écran) alors que `td` est une cellule de données. `thead` et `tbody` séparent l'en-tête du corps, ce qui permet de les styler différemment et de répéter l'en-tête à l'impression.

3.4 Section « Inscription » (id="inscription")

Le formulaire permet de s'inscrire à une activité. Il contient **neuf champs de types différents**, au-delà des six exigés :

Champ	Élément / type	Remarques
Nom complet	`input type="text"`	Obligatoire (`required`, `autocomplete="name"`)
Adresse électronique	`input type="email"`	Obligatoire ; format vérifié par le navigateur
Téléphone	`input type="tel"`	Clavier numérique sur mobile
Date de participation	`input type="date"`	Sélecteur de date natif
Filière	`input + datalist`	Cinq suggestions (GIIA, Génie informatique, Génie civil, Génie énergétique, Cybersécurité), saisie libre possible
Club souhaité	`select / option`	Liste déroulante des quatre clubs, avec une option vide « Choisir un club »
Niveau d'étude	`input type="radio"`	Trois choix groupés dans `fieldset` + `legend` ; 1re année cochée par défaut
Commentaire	`textarea`	Facultatif ; texte d'exemple en `placeholder`
Conditions	`input type="checkbox"`	Obligatoire (`required`)
Envoi	`button type="submit"`	Valide puis affiche une confirmation

Chaque champ est relié à son libellé par `label for="…"` et l'`id` correspondant : cliquer sur le libellé active le champ, et un lecteur d'écran annonce le libellé. Les valeurs `name` sont cohérentes (`nom_complet`, `email`,

`telephone`…). Le `placeholder` n'est utilisé que comme indication facultative (« Commencez à saisir », « Besoins particuliers, questions… »), jamais à la place d'un libellé. Les champs obligatoires sont signalés par un astérisque rouge expliqué en tête de section. En CSS, un champ invalide après saisie prend une bordure rouge (`input:user-invalid`) et le champ actif reçoit un contour bleu. La grille à deux colonnes passe sur une seule colonne sur mobile.

```
<div class="champ">
  <label for="email">Adresse électronique <span class="req">*</span></label>
  <input type="email" id="email" name="email" autocomplete="email" required>
</div>
<fieldset class="large">
  <legend>Niveau d'étude</legend>
  <label for="n1"><input type="radio" id="n1" name="niveau" value="1" checked> 1<sup>re</sup> année</label>
  ...
</fieldset>
```

Un court script empêche l'envoi réel (il n'y a pas de serveur), récupère le nom saisi, vide le formulaire et affiche un message de confirmation personnalisé (« Merci … , votre inscription est enregistrée. ») dans un élément `output` avec `aria-live="polite"`, de sorte que le message soit aussi annoncé aux lecteurs d'écran. Le navigateur a déjà contrôlé les champs `required` avant que le script ne s'exécute.

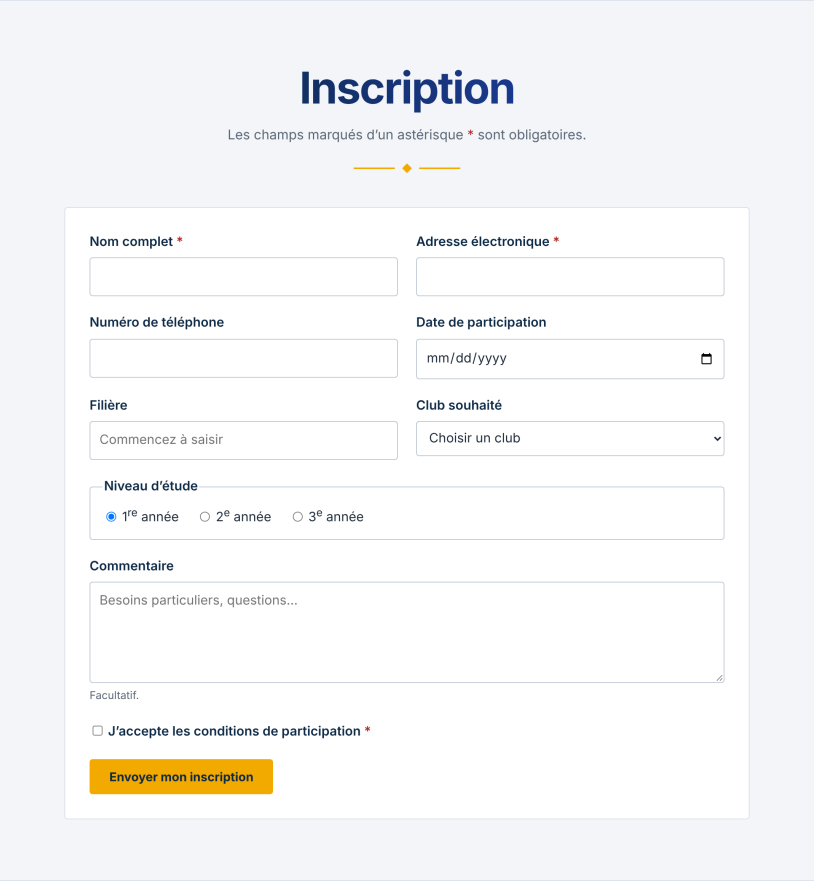


Figure 6 – Formulaire d'inscription.

3.5 Section « Ressource intégrée » (id="integration")

Cette section intègre une **carte OpenStreetMap** dans la page grâce à un élément `iframe`, qui affiche une page externe dans un cadre. L'adresse utilisée est celle de l'intégration officielle (`export/embed.html`) : le paramètre `bbox` définit la zone visible autour de l'ENSA Safi, `layer=mapnik` choisit le fond de carte standard et `marker=32.32649,-9.26442` (latitude, longitude) place un **marqueur sur l'ENSA Safi**. Le style CSS étire le cadre à toute la largeur de la section avec une hauteur de 360 px.

Sous la carte, plusieurs liens classiques restent disponibles si l'intégration échoue : « Voir ENSA Safi sur la grande carte » (lien OpenStreetMap avec `mlat` et `mlon` pour recentrer sur le marqueur), « Ouvrir

OpenStreetMap dans un nouvel onglet » et un bouton « Ouvrir la carte sur Google Maps » pour obtenir l'itinéraire. Les liens externes s'ouvrent dans un nouvel onglet avec `rel="noopener"`.

```
<iframe width="425" height="350"
  src="https://www.openstreetmap.org/export/embed.html?bbox=-9.2669%2C32.3249%2C-9.2619%2C32.3280
       &layer=mapnik&marker=32.32649%2C-9.26442"
  style="border: 1px solid black"></iframe>
<small><a href="https://www.openstreetmap.org/?mlat=32.32649&mlon=-9.26442#map=18/32.32649/-9.26442">
  Voir ENSA Safi sur la grande carte</a></small>
<a class="btn" href="https://www.google.com/maps/search/?api=1&query=ENSA+Safi+Maroc"
   target="_blank" rel="noopener">Ouvrir la carte sur Google Maps</a>
```

Limite constatée. Lors des premiers tests, l'intégration de la carte a affiché le message « Ce contenu est bloqué ». Un site peut en effet interdire d'être affiché dans une `iframe` d'un autre site (en-têtes `X-Frame-Options` ou `Content-Security-Policy: frame-ancestors`). Ce blocage ne doit jamais être contourné : on utilise l'adresse d'intégration que le service propose officiellement et on conserve des liens externes de secours. La carte est chargée depuis un serveur externe : elle n'apparaît que si l'ordinateur est connecté à Internet, et il vaut mieux tester la page via un petit serveur local (extension « Live Server » de VS Code, par exemple) plutôt qu'en ouvrant le fichier depuis le disque. L'intégration OpenStreetMap n'affiche qu'un marqueur, sans texte ; pour une bulle portant le nom de l'école, il faudrait une bibliothèque cartographique comme Leaflet.

3.6 Section « Médias » (id="medias")

La section propose un lecteur audio et un lecteur vidéo. L'attribut `controls` affiche les commandes natives (lecture, pause, volume, plein écran pour la vidéo). Chaque lecteur contient un élément `source` avec le chemin et le type MIME du fichier ; le navigateur choisit le format qu'il sait lire. Le texte placé entre les balises est le **texte de remplacement** : il s'affiche si le navigateur ne gère pas l'élément, avec un lien de téléchargement pour l'audio. `preload="none"` (audio) et `preload="metadata"` (vidéo) évitent de télécharger inutilement les fichiers au chargement.

```
<audio controls preload="none">
  <source src="media/extrait.mp3" type="audio/mpeg">
  Votre navigateur ne prend pas en charge l'audio.
  <a href="media/extrait.mp3">Télécharger l'extrait</a>.
</audio>
<video controls preload="metadata" width="640">
  <source src="media/presentation.mp4" type="video/mp4">
  Votre navigateur ne prend pas en charge la vidéo. <a href="media/presentation.mp4">Télécharger la vidéo</a>.
</video>
```

Fichiers attendus : `media/extrait.mp3` et `media/presentation.mp4`. Les chemins sont relatifs au fichier `index.html` : le dossier `media` doit donc se trouver à côté de lui.

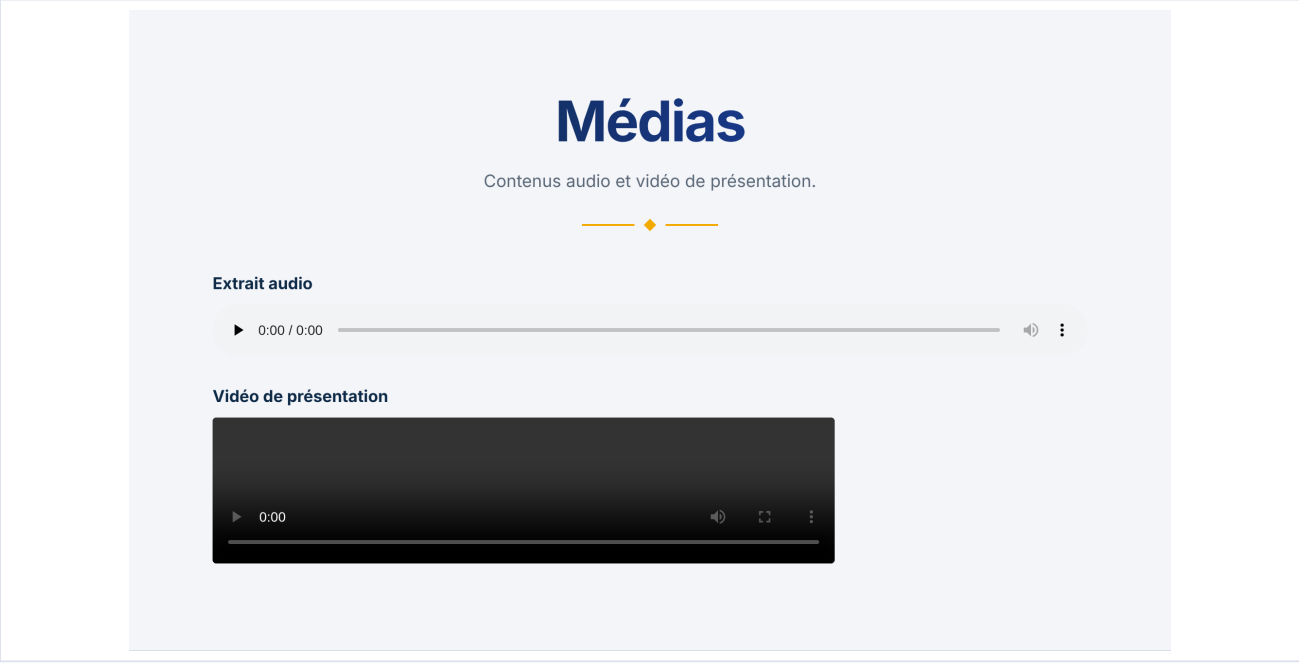


Figure 7 – Lecteurs audio et vidéo (les fichiers ne sont pas fournis dans cette capture).

Média local ou vidéo intégrée. Un média local est un fichier hébergé avec la page, lu directement par le navigateur avec `video` : on maîtrise le fichier mais on supporte l'hébergement et la bande passante. Une vidéo de plateforme (YouTube, par exemple) est affichée dans une `iframe` : le lecteur, l'hébergement et le format dépendent de la plateforme, qui peut aussi ajouter publicité ou suivi.

3.7 Section « Canvas » (id="dessin")

L'élément `canvas` (500 × 220 pixels) est une surface de dessin vide : tout le contenu est produit par JavaScript. `getContext("2d")` renvoie l'objet qui expose les méthodes de dessin en deux dimensions. Le script s'exécute après l'événement `DOMContentLoaded`, ce qui garantit que l'élément existe quand on dessine. Le dessin comporte cinq éléments : un rectangle bleu, un cercle safran cerclé de bleu nuit, une ligne, un triangle vert (forme supplémentaire) et deux lignes de texte. Les attributs `role="img"` et `aria-label` décrivent le dessin aux technologies d'assistance, puisqu'un canvas n'a pas de contenu textuel.

```
const ctx = document.getElementById("zoneDessin").getContext("2d");
ctx.fillStyle = "#2446C9"; ctx.fillRect(20, 20, 150, 70);                       // rectangle
ctx.beginPath(); ctx.arc(250, 70, 40, 0, 2 * Math.PI);                         // cercle
ctx.fillStyle = "#F2A900"; ctx.fill(); ctx.lineWidth = 3; ctx.strokeStyle = "#0E2A47"; ctx.stroke();
ctx.beginPath(); ctx.moveTo(330, 30); ctx.lineTo(470, 100); ctx.stroke();      // ligne
ctx.beginPath(); ctx.moveTo(380, 190); ctx.lineTo(430, 120); ctx.lineTo(480, 190);
ctx.closePath(); ctx.fillStyle = "#1F8A5B"; ctx.fill();                         // triangle
ctx.fillStyle = "#0E2A47"; ctx.font = "600 20px sans-serif";
ctx.fillText("Portail étudiant HTML5", 20, 150);                                // texte
```

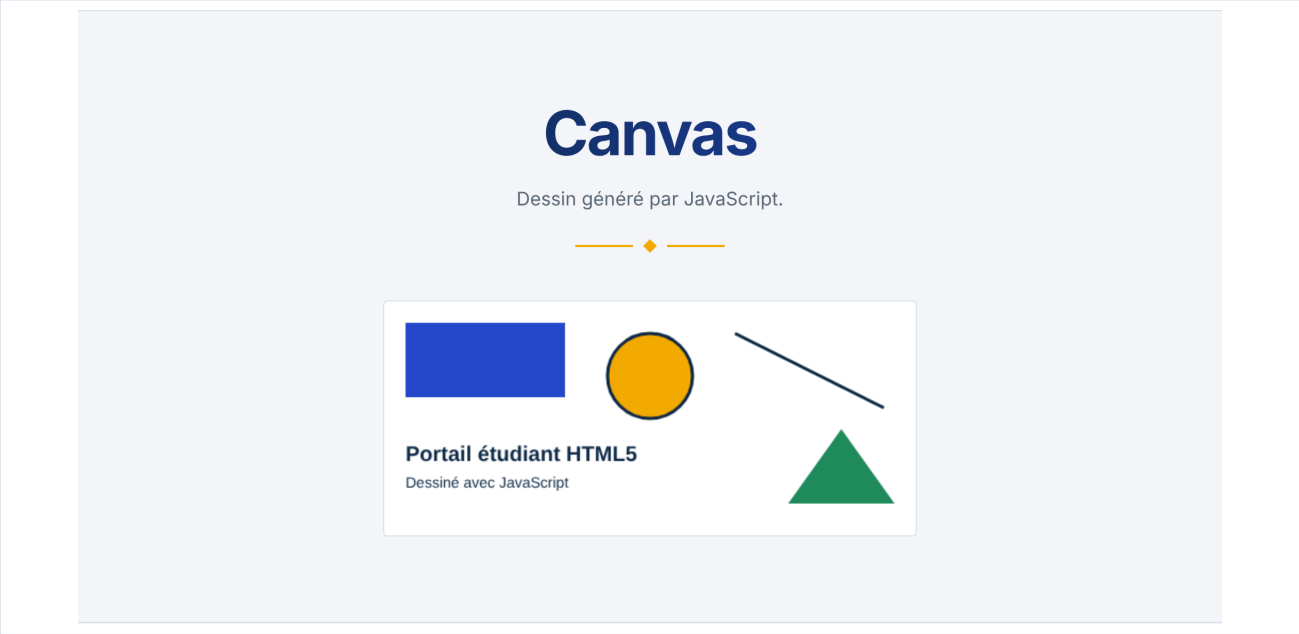


Figure 8 – Dessin produit par JavaScript sur le canvas.

3.8 Section « Compléments » (id="complements")

Cette section regroupe des éléments découverts sur la documentation MDN, au-delà des trois exigés. Chacun est accompagné d'un commentaire HTML expliquant son utilité.

Élément	Utilité	Exemple dans la page
`meter`	Mesure dans une plage connue (`min`, `max`, `low`, `high`, `optimum`)	36 places occupées sur 50 au Club Robotech
`progress`	Avancement d'une tâche	Préparation de l'événement avancée à 65 %
`details` / `summary`	Bloc que l'utilisateur ouvre ou ferme, sans JavaScript	Questions fréquentes (gratuité, modification de la date)
`output`	Résultat d'un calcul ou d'une action	Simulateur d'heures de bénévolat : 2 points par heure ; message de confirmation du formulaire
`address`	Coordonnées de contact	Adresse, e-mail et téléphone de l'établissement, avec icônes SVG
`time`	Date ou heure lisible par une machine	Dates du planning
`datalist`	Suggestions de saisie dans un champ libre	Liste des filières

Le **simulateur** associe un curseur `input type="range"` (de 0 à 10 heures) à deux éléments `output` : le premier affiche la durée choisie, le second calcule les points gagnés (durée × 2) grâce à l'attribut `oninput` du formulaire. Le bloc **Contact** est un élément `address` : l'adresse (Route Sidi Bouzid BP 63, 46000 Safi, Maroc), un lien `mailto:` (ensas@uca.ac.ma) et un lien `tel:` (+212 656 200007) précédés d'icônes SVG intégrées, masquées aux lecteurs d'écran par `aria-hidden="true"`.

```
<meter min="0" max="50" low="10" high="40" optimum="25" value="36">36 sur 50</meter>
<progress max="100" value="65">65 %</progress>
<form oninput="resultat.value = (duree.valueAsNumber * 2) + ' points'" onsubmit="return false">
  <label for="duree">Durée : <output name="heures" id="heures">4</output> h</label>
  <input type="range" id="duree" name="duree" min="0" max="10" value="4" oninput="heures.value = this.value">
  <p>Points gagnés : <output name="resultat" for="duree">8 points</output></p>
</form>
```

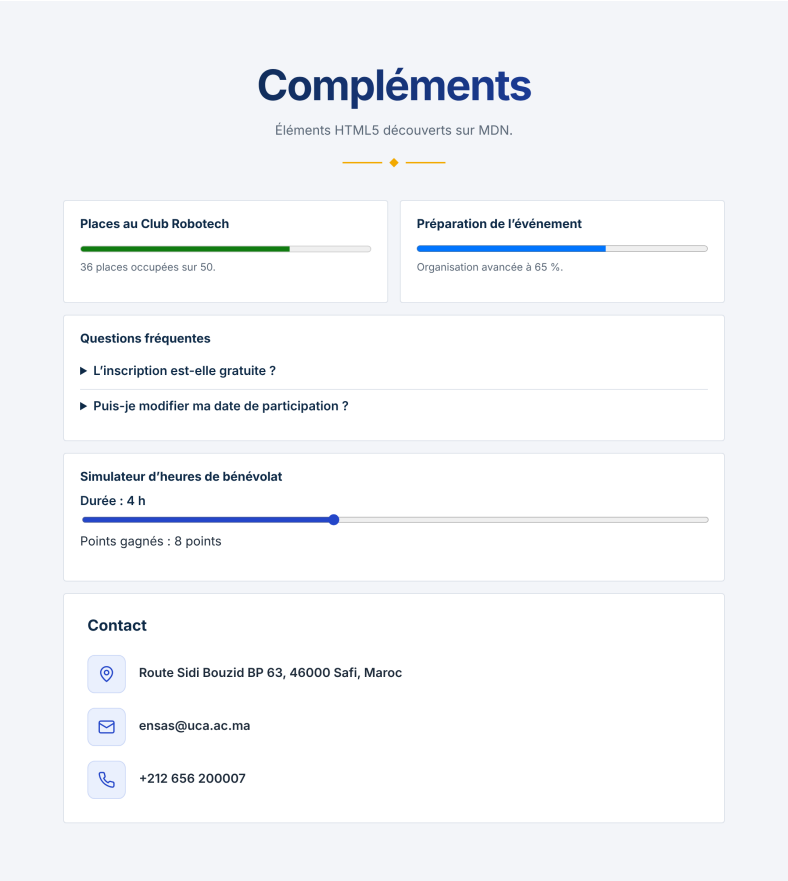


Figure 9 – Section « Compléments » : meter, progress, details, output et address.

3.9 Pied de page

Le `footer` rappelle le cadre de réalisation de la page : TD-TP HTML5, module Système de Gestion de Contenu, ENSA Safi, 2026-2027.

4 Tests et liste de vérification

Points contrôlés avant la remise du travail.

Point de contrôle	Résultat	Observation
`<!DOCTYPE html>` et `<html lang="fr">`	Conforme	Présents en tête de document
Titre de l'onglet pertinent, UTF-8 déclaré	Conforme	Titre « Portail étudiant : activités et ressources | ENSA Safi »
Menu : bouton, tiroir et liens vers des identifiants existants	Conforme	7 liens du menu + bouton d'en-tête, aucun lien cassé, aucun identifiant en double ; fermeture au clic, sur le fond et avec Échap
Tableau : en-tête et au moins 4 lignes	Conforme	`thead` + 4 lignes dans `tbody`
Libellé explicite pour chaque champ	Conforme	Chaque `label for` correspond à un `id`
Champs obligatoires indiqués	Conforme	`required` + astérisque (nom, e-mail, conditions)
iframe : carte avec marqueur et liens de secours	Conforme	Marqueur sur l'ENSA Safi ; trois liens externes de secours. Un attribut `title` descriptif reste à ajouter sur l'iframe

Point de contrôle	Résultat	Observation
Chemins des médias corrects, fichiers disponibles	À vérifier	Placer extrait.mp3 et presentation.mp4 dans media/
Canvas : plusieurs formes	Conforme	Rectangle, cercle, ligne, triangle, texte
Éléments supplémentaires compréhensibles	Conforme	Sept éléments, chacun commenté
Code indenté et testé dans un navigateur	Conforme	Test sur ordinateur et sur écran de 390 px

Tests manuels effectués : ouverture et fermeture du menu tiroir (bouton, fond, touche Échap), clic sur chaque lien du menu, envoi du formulaire vide (le navigateur signale les champs manquants), envoi du formulaire rempli (message de confirmation), ouverture des blocs `details`, déplacement du curseur du simulateur, vérification de l'affichage du canvas après chargement et réduction de la fenêtre pour contrôler l'adaptation mobile.

5 Démonstration : réponses aux questions

Question 1 – Quelle différence existe-t-il entre iframe, video et canvas ?

Ces trois éléments affichent du contenu « riche » dans une page, mais ils n'ont ni la même source ni le même rôle.

	iframe	video	canvas
Rôle	Afficher une autre page Web dans un cadre	Lire un fichier vidéo	Dessiner des pixels générés par un script
Origine du contenu	Un site externe (ou une autre page du site)	Un fichier (mp4, webm…)	Du code JavaScript
Interface fournie	Celle de la page intégrée	Commandes natives avec `controls`	Aucune : tout est à programmer
JavaScript nécessaire	Non	Non (pour une lecture simple)	Oui, sans lui le canvas reste vide
Accessibilité	`title` recommandé	Sous-titres, texte de remplacement	`aria-label` ou contenu de repli
Point de vigilance	Le site peut interdire l'intégration ; sécurité à contrôler (`sandbox`)	Formats et poids des fichiers	Le résultat est une image : le texte dessiné n'est pas sélectionnable
Exemple dans la page	Carte OpenStreetMap de l'ENSA Safi	Vidéo de présentation	Dessin de formes

En résumé : `iframe` emprunte une page existante, `video` lit un fichier multimédia et `canvas` fabrique une image par programmation.

Question 2 – Quels éléments facilitent l'accessibilité de la page ?

L'accessibilité vise à rendre la page utilisable par tous, y compris les personnes qui utilisent un lecteur d'écran, un clavier seul ou un zoom important. Dans cette page :

- **La langue :** `lang="fr"` permet au lecteur d'écran de choisir la bonne prononciation.

- **La structure sémantique :** `header`, `nav`, `main`, `section`, `footer` et une hiérarchie de titres (`h1`, `h2`, `h3`) permettent de naviguer par repères.
- **Le menu tiroir :** le bouton annonce son état avec `aria-expanded` et le menu qu'il commande avec `aria-controls` ; le menu fermé est masqué (`visibility: hidden`) ; la touche Échap le ferme ; les icônes décoratives portent `aria-hidden="true"`.
- **Les alternatives textuelles :** `aria-label` sur le canvas, texte de remplacement dans `audio` et `video`, liens de secours sous la carte.
- **Les formulaires :** `label for`, `fieldset` et `legend` relient chaque champ à son intitulé ; les champs obligatoires sont signalés par `required` ; `aria-live` annonce le message de confirmation.
- **Le tableau :** `caption`, `th` et `scope` permettent d'associer chaque donnée à son en-tête.
- **La navigation au clavier :** le style `:focus-visible` rend le focus bien visible ; les éléments natifs (`details`, `button`) sont utilisables au clavier.
- **La lisibilité :** contrastes élevés entre texte et fond, tailles relatives et prise en compte de `prefers-reduced-motion` pour limiter les animations.

Une amélioration possible : ajouter un attribut `title` descriptif à l'`iframe` (par exemple « Carte OpenStreetMap indiquant l'emplacement de l'ENSA Safi »), sans quoi elle n'est annoncée que comme « cadre ».

Question 3 – La validation HTML d'un formulaire garantit-elle que les données sont exactes et fiables ?

Non. La validation HTML contrôle uniquement la forme des données, pas leur vérité.

- **Elle vérifie le format, pas l'existence.** Un champ `type="email"` accepte a@b.c alors que cette adresse peut ne pas exister ou ne pas appartenir à la personne. De même, `type="tel"` n'impose aucun format de numéro et `type="date"` accepte une date passée ou absurde si on ne fixe pas `min` et `max`.
- **Elle s'exécute dans le navigateur de l'utilisateur.** Celui-ci peut la désactiver, modifier le code avec les outils de développement ou envoyer directement une requête au serveur sans passer par le formulaire.
- **Elle ne détecte pas les mensonges ni les erreurs de saisie :** un nom, un niveau d'étude ou un choix de club faux restent valides.

Pour des données fiables, il faut donc : (1) revalider côté serveur toutes les données reçues, (2) assainir les entrées pour se protéger des injections, (3) confirmer les informations importantes, par exemple par un e-mail de vérification, et (4) imposer des contraintes supplémentaires (`pattern`, `minlength`, `min`/`max`). La validation HTML reste utile : elle améliore l'expérience en signalant tout de suite les oublis, mais elle ne remplace jamais le contrôle côté serveur.

Question 4 – Deux éléments HTML découverts grâce à MDN

`<details>` **et** `<summary>`

`details` crée un bloc repliable : `summary` en est le titre cliquable et le reste du contenu apparaît à l'ouverture. Le comportement est natif : pas de JavaScript, utilisable au clavier et compris par les lecteurs d'écran. Dans la page, il sert à présenter les questions fréquentes sans surcharger l'écran.

```
<details>
  <summary>L'inscription est-elle gratuite ?</summary>
  <p>Oui, toutes les activités sont gratuites pour les étudiants de l'établissement.</p>
</details>
```

`<meter>`

`meter` représente une mesure à l'intérieur d'une plage connue, grâce aux attributs `min`, `max`, `low`, `high` et `optimum` : le navigateur change la couleur de la barre selon la zone. Il se distingue de `progress`, qui

représente l'avancement d'une tâche vers son achèvement. Dans la page, il indique que 36 places sur 50 sont occupées au Club Robotech.

```
<meter min="0" max="50" low="10" high="40" optimum="25" value="36">36 sur 50</meter>
```

6 Difficultés rencontrées et conclusion

Difficultés rencontrées

La principale difficulté a concerné l'iframe : lors des premiers tests, la carte a affiché « Ce contenu est bloqué ». Cela nous a appris qu'un site peut refuser d'être intégré dans la page d'un autre, qu'il faut utiliser l'adresse d'intégration officielle du service et qu'il faut toujours prévoir des liens de secours ; nous avons aussi ajouté un marqueur pour indiquer précisément l'ENSA Safi sur la carte.

Le menu tiroir a demandé de coordonner le HTML, le CSS et le JavaScript : la classe `menu-open` du `body`, les attributs ARIA, le masquage du menu fermé et la fermeture par la touche Échap. Nous avons également dû veiller à ce que les chemins des fichiers du dossier `media/` correspondent à l'arborescence réelle, à associer correctement chaque `label` à l'`id` de son champ, et à dessiner le canvas seulement après le chargement du document. Enfin, la documentation MDN nous a permis de découvrir `details`, `meter`, `output` et `datalist`, que nous ne connaissions pas.

Conclusion

Ce TD-TP a permis de construire une page Web complète qui combine structure sémantique, menu de navigation, tableau, formulaire, ressource intégrée, multimédia, dessin dynamique et éléments complémentaires. Au-delà de la maîtrise de chaque balise, il met en évidence trois principes : choisir l'élément HTML qui correspond au sens du contenu, penser à l'accessibilité dès la conception, et ne jamais considérer la validation côté navigateur comme une garantie de fiabilité.

Réalisé par	**RACHAD MUSTAPHA** **AZIZ YASSINE**